

Data Structures and Algorithms – IT1170

Practical 6 – Trees / Answers

Exercise 1

➤ Q1 & Q2 - Node Class

```
class Node {  
  
    int empNo;  
    String name;  
  
    Node leftChild;  
    Node rightChild;  
  
    public Node(int empNo, String name) {  
  
        this.empNo = empNo;  
        this.name = name;  
    }  
  
    public void displayNode() {  
  
        System.out.println(  
            "Employee No: "  
            + empNo +  
            ", Name: "  
            + name);  
    }  
}
```

➤ Q3 - Tree Class

Basic Structure

```
class Tree {  
  
    Node root;  
  
    public Tree() {  
  
        root = null;  
    }  
}
```

1. Find Method

```
public Node find(int emp) {  
  
    Node current = root;  
  
    while(current != null) {  
  
        if(emp == current.empNo)  
            return current;  
  
        else if(emp < current.empNo)  
            current = current.leftChild;  
  
        else  
            current = current.rightChild;  
    }  
  
    return null;  
}
```

2. Insert Method

```
public void insert(  
    int emp,  
    String name)  
{  
    Node newNode =  
        new Node(emp, name);  
  
    if(root == null) {  
  
        root = newNode;  
        return;  
    }  
  
    Node current = root;  
    Node parent;  
  
    while(true) {  
  
        parent = current;  
  
        if(emp < current.empNo) {  
  
            current =
```

```

        current.leftChild;

        if(current == null) {

            parent.leftChild =
                newNode;

            return;
        }
    }
    else {

        current =
            current.rightChild;

        if(current == null) {

            parent.rightChild =
                newNode;

            return;
        }
    }
}

```

3. Inorder Traversal

Left → Root → Right

```

public void inOrder(Node localRoot)
{
    if(localRoot != null)
    {
        inOrder(
            localRoot.leftChild);

        localRoot.displayNode();

        inOrder(
            localRoot.rightChild);
    }
}

```

4. Preorder Traversal

Root → Left → Right

```
public void preOrder(Node localRoot)
{
    if(localRoot != null)
    {
        localRoot.displayNode();

        preOrder(
            localRoot.leftChild);

        preOrder(
            localRoot.rightChild);
    }
}
```

5. Post order Traversal

Left → Right → Root

```
public void postOrder(Node localRoot)
{
    if(localRoot != null)
    {
        postOrder(
            localRoot.leftChild);

        postOrder(
            localRoot.rightChild);

        localRoot.displayNode();
    }
}
```

➤ Q4 - Recursive Find Method

```
public Node findRecursive(
    Node current,
    int emp)
{
    if(current == null)
        return null;

    if(current.empNo == emp)
```

```
        return current;

    if(emp < current.empNo)
        return findRecursive(
            current.leftChild,
            emp);

    return findRecursive(
        current.rightChild,
        emp);
}
```

Usage:

```
Node found =
    tree.findRecursive(
        tree.root,
        149);
```

➤ Q5 - deleteAll()

```
public void deleteAll()
{
    root = null;
}
```

➤ Q6 - Main Application

```
import java.util.Scanner;

public class TreeTest {

    public static void main(String[] args) {

        Tree tree = new Tree();

        tree.insert(149, "Anusha");
        tree.insert(167, "Kosala");
        tree.insert(47, "Dinusha");
        tree.insert(66, "Mihiri");
        tree.insert(159, "Jayani");
        tree.insert(118, "Nimal");
        tree.insert(195, "Nishantha");
        tree.insert(34, "Avodya");
        tree.insert(105, "Bimali");
        tree.insert(133, "Sampath");

        System.out.println(
            "INORDER");

        tree.inOrder(tree.root);

        System.out.println(
            "\nPREORDER");

        tree.preOrder(tree.root);

        System.out.println(
            "\nPOSTORDER");

        tree.postOrder(tree.root);

        Scanner input =
            new Scanner(System.in);

        System.out.print(
            "\nEnter Employee No: ");

        int emp =
            input.nextInt();

        Node found =
            tree.findRecursive(
```

```
        tree.root,
        emp);

    if(found != null)
        found.displayNode();
    else
        System.out.println(
            "Employee Not Found");

    tree.deleteAll();

    System.out.println(
        "\nTree Deleted");

    tree.inOrder(tree.root);
}
}
```